Training Neural Networks : Initialization

1. What is Initialization ?

Initialization means choosing the starting values of weights (and biases) before training a neural network.

Good initialization helps the network to learn faster and prevents problems like vanishing / exploding gradients.

2. Why not initialize all weights to zero ?

→ If all weights are zero, all neurons in a layer will produce the same output.

→ They will get same gradients during backpropagation and learn the same thing.

This is called the symmetry problem.

3. Random Initialization

Weights are initialized with small random values instead of zeros.

W = random values

Example : −0.04, 0.01, 0.18, −0.12, 0.30

It breaks symmetry problem.

4. Initialization using Distributions

(a) Uniform Distribution $W \sim U(-a, a)$

- Weights are randomly chosen from a uniform distribution between −a and +a.
- Here, 'a' controls the range.
- Values can be positive or negative.

Example : −0.40, 0.18, +0.12, 0.30

(b) Normal (Gaussian) Distribution $W \sim N(0, \sigma^2)$

- Weights are sampled from normal distribution with mean 0 and variance σ^2.
- Most values lie near 0.
- Here, σ^2 controls the spread.

Example : 0.09, −0.11, 0.8, −0.2

5. Xavier / Glorot Initialization

(used with sigmoid / tanh activation functions)

Uniform version :

$$W \sim U\left(-\sqrt{\frac{6}{n_{in}+n_{out}}}, \sqrt{\frac{6}{n_{in}+n_{out}}}\right)$$

where, n_{in} = number of input neurons

n_{out} = number of output neurons

Normal version : $W \sim N\left(0, \frac{2}{n_{in}+n_{out}}\right)$

Advantages :

- Faster convergence.
- Helps keep the variance of activations and gradients stable across layers.
- Reduces vanishing and exploding gradient problems.

Disadvantages :

- Not ideal for ReLU activations.
- May not work well for very deep ReLU-based networks.

Remember : Xavier → sigmoid, tanh

6. He / Kaiming Initialization

(exclusively used for ReLU and its variants)

Normal version : $W \sim N\left(0, \frac{2}{n_{in}}\right)$

Uniform version : $W \sim U\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right)$

Advantages :

- Designed for ReLU.
- Prevents vanishing gradient problem.
- Keeps variance of activations approximately constant.
- Most widely used initialization for modern deep networks.

Disadvantages :

- Not suitable for sigmoid / tanh.

Remember : He → ReLU

7. LeCun Initialization

(used with SELU activation function)

Normal version : $W \sim N\left(0, \frac{1}{n_{in}}\right)$

Uniform version : $W \sim U\left(-\sqrt{\frac{3}{n_{in}}}, \sqrt{\frac{3}{n_{in}}}\right)$

Advantages :

- Works well with SELU.
- Helps maintain self-normalizing property.
- Keeps activations in a good range.

Disadvantages :

- Not preferred for ReLU networks.
- Mainly useful when using SELU.

Remember : LeCun → SELU

8. Orthogonal Initialization

- Weights are initialized using orthogonal matrix.
- Helps preserve the magnitude of signals across layers.

Advantages :

- Good for deep networks and RNNs.
- Helps in stable and fast convergence.

Disadvantages :

- Slightly more computationally expensive.
- Not necessary for all networks.

9. Sparse Initialization

- Most weights are initialized to zero and only a small fraction are random non-zero values.

0	0	0.02	0	-0.03	0	0

Advantages :

- Fewer active connections initially.
- Can reduce computation.

Disadvantages :

- Too much sparsity can slow down learning.
- Not commonly used in practice today.

10. Other (Less Common)

- Truncated Normal Initialization (sample from normal distribution and cut off values beyond a range)
- Variance Scaling Initialization (general form used in many frameworks)
- Identity Initialization (used in some specific architectures like RNNs, ResNets)

11. Bias Initialization

- Biases are mostly initialized to zero. $b = 0$
- Or a small positive value can be used, e.g. $b = 0.01$ especially with ReLU.

Key Point :
Good initialization → Better learning faster and reduces overfitting.

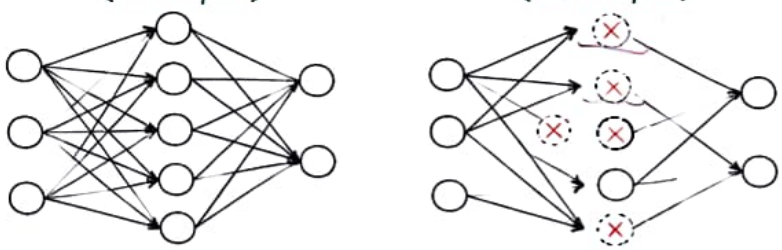
DROPOUT

Dropout is a regularization technique in which randomly selected neurons are temporarily deacticated during training to reduce overfitting and improve generalization.

1. What is Dropout ?

→ During training, dropout randomly "drops" (switches off) some neurons with a certain probability.

(No Dropout) (With Dropout)



Active neuron
Dropped neuron (during training)

→ Dropped neurons are ignored for that training step.

2. Why do we need Dropout ?

→ Without dropout, neurons can become highly co-adapted on training data.
→ The network may memorize the training data (overfitting).
→ Dropout prevents this by making the network learn more robust and general features.

Overfitting
↓
Training accuracy → High
Test accuracy → Low

3. How does Dropout work ?

- Each neuron is kept with probability q and dropped with probability p.
- Dropout rate p = probability of dropping a neuron.
- Typically $p = 0.2, 0.3, 0.5$ (i.e. 20%, 30%, 50%).

$q = 1 - p$ → probability of keeping a neuron.

Example :
If $p = 0.5$, then each neuron has 50% chance of being dropped during training.

4. During Training vs During Testing

During Training (Dropout ON)

- Some neurons are randomly dropped.
- Different neurons are dropped in each training step.
- Network learns using many different sub-networks.

For each training step :
Input → Layer → Dropout → Output

During Testing / Inference (Dropout OFF)

- All neurons are used.
- No neuron is dropped.
- We use the full network for predictions.

During testing :
Input → Layer → Output

Important :
Dropout is used ONLY during training.

5. Dropout Mechanism (Inverted Dropout)

- If a neuron is dropped → its output is set to 0.
- If a neuron is kept → its output is scaled up to keep the expected value same.
- Suppose a neuron output is h.
  - * If neuron is dropped: $h' = 0$
  - * If neuron is kept: $h' = \frac{h}{1-p}$ (i.e. scaled by $\frac{1}{1-p}$)

Why scaling ?
So that the overall magnitude of activations remains approximately the same during training and testing.

6. Advantages of Dropout

1. Reduces overfitting.
2. Improves generalization on unseen data.
3. Prevents complex co-adaptations between neurons.
4. Acts like training an ensemble of many sub-networks.
5. Easy to implement.

7. Disadvantages of Dropout

1. Training becomes slower (convergence may be slow).
2. If dropout rate is too high → may cause underfitting.
3. Not suitable for small networks or small datasets (may lose too much information).

8. Key Points to Remember

- Dropout is a regularization technique.
- Used only during training.
- Typical dropout rate $p = 0.2$ to 0.5.
- At test time, use the full network (without dropout).
- Helps reduce overfitting and improves generalization.

9. Where is Dropout Applied ?

- Usually applied to hidden layers (not to input layer).
- Can be used in fully connected layers and convolutional layers.
- Common in deep neural networks, CNNs, RNNs, Transformers.

10. Summary

Dropout randomly switches off neurons during training, forcing the network to learn robust features without depending on any particular neuron. During testing, all neurons are used to make predictions.

BATCH NORMALIZATION AND DROPOUT

★ Important

Both are used during training neural networks.

1. BATCH NORMALIZATION (BatchNorm)

→ Batch Normalization is a technique that normalizes the activations of a layer during training.

It keeps the values going into the next layer at a reasonable scale and makes training faster and more stable.

2. Why do we need Batch Normalization ?

- The distribution of inputs to a layer keeps changing during training.
- This can slow down training and make optimization difficult.
- BatchNorm reduces this problem.

3. How Batch Normalization works ?

Suppose a mini-batch of activations : $x = [2, 4, 6, 8]$

① Compute mean (μ_B) :

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i = \frac{2+4+6+8}{4} = \frac{20}{4} = 5$$

② Compute variance (σ_B^2) :

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

$$= \frac{[(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2]}{4}$$

$$= \frac{9+1+1+9}{4} = \frac{20}{4} = 5$$

③ Normalize :

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \quad (\epsilon \text{ is a small number e.g. } 10^{-5})$$

If $\epsilon \approx 0$, $\sqrt{\sigma_B^2} = \sqrt{5} \approx 2.236$

For $x = [2, 4, 6, 8] \rightarrow \hat{x}_i =$ $[-1.342, -0.447, 0.447, 1.342]$

④ Scale and Shift (learnable parameters γ and β):

$$y_i = \gamma \hat{x}_i + \beta$$

$\gamma \rightarrow$ scale (learned)

$\beta \rightarrow$ shift (learned)

Final output after BatchNorm is : $y = \gamma \hat{x} + \beta$

4. Flow in a neural network

Linear layer → BatchNorm → Activation (ReLU) → Next layer

5. Training vs Testing

During Training	During Testing
• Uses mean (μ_B) and variance (σ_B^2) computed from the current mini-batch.	• Uses the running mean and running variance collected during training.
• Also keeps running estimates : running mean, running variance	• No batch statistics are used.

6. Advantages

- Faster convergence
- More stable training
- Helps in gradient flow
- Allows larger learning rates
- Acts as a mild regularizer (can reduce overfitting a little)

7. Disadvantages

- Depends on batch statistics (small batch size may be less effective).
- Adds extra computation.
- Behavior differs between training and testing.

1. DROPOUT

→ Dropout is a regularization technique that randomly sets some neuron activations to zero during training to reduce overfitting.

2. Why do we need Dropout ?

- Without dropout, the network may become too dependent on some neurons.
- Dropout forces the network to learn using different combinations of neurons.
- Main purpose : Reduce Overfitting.

3. How Dropout works ?

- For each neuron, with probability p it is dropped (set to zero) and with probability (1 − p) it is kept.
- p = dropout probability. (Example : p = 0.5)
- Implemented as multiplying with a mask D.

Inverted Dropout (most commonly used)

$$y = \frac{(D * x)}{1-p} \quad \text{where} \quad D = \begin{cases} 0 & \text{with prob. } p \\ 1 & \text{with prob. } 1-p \end{cases}$$

4. Example (Inverted Dropout)

Suppose activations of a layer (before dropout):

$x = [2, 4, 6, 8, 10]$, $p = 0.4$

So, probability of keeping = $1 - p = 0.6$

During one training step, assume the mask sampled is

$D = [1, 0, 1, 0, 1]$

Apply inverted dropout :

$$y = \frac{(D * x)}{1-p} = \frac{(D * x)}{0.6}$$

$D * x = [2, 0, 6, 0, 10]$

$$y = \left[\frac{2}{0.6}, 0, \frac{6}{0.6}, 0, \frac{10}{0.6}\right]$$

$y = [3.33, 0, 10, 0, 16.67]$

Why divide by (1 − p)?

It keeps the expected value (average magnitude) the same as before dropout.

Check (approx) :

Original mean = $\frac{2+4+6+8+10}{5} = 6$

After inverted dropout :

$$E(y) \approx 0.6 * \left(\frac{2}{0.6} + \frac{6}{0.6} + \frac{\frac{10}{0.6} + 0 + \left(\frac{10}{0.6}\right)}{5}\right) = 6 \quad ✓$$

5. During Training vs During Testing

During Training (Dropout ON)	During Testing / Inference (Dropout OFF)
• Random neurons are dropped.	• All neurons are used.
• Different neurons are dropped in each training step.	• No neuron is dropped.
• Use inverted dropout (scale by $1/(1-p)$).	• No scaling is needed.
	• We use the full network for predictions.

6. Advantages

- Reduces overfitting
- Improves generalization
- Prevents co-adaptation of neurons
- Easy to implement
- Acts like training many subnetworks (ensemble effect)

7. Disadvantages

- Training becomes slower (may require more epochs)
- If dropout rate is too high → underfitting
- Not always necessary, depends on problem

Summary Table

Feature	Batch Normalization	Dropout
Main Goal	Stable & faster training	Reduce overfitting
What it does	Normalizes activations	Randomly drops activations
When used	During training & testing (with running stats)	Only during training
Has learnable parameters?	Yes (γ, β)	No
Effect on training	Faster, more stable	More robust, less overfitting
Testing behavior	Uses running mean & var	All neurons active, no scaling

OVERFITTING

1. What is Overfitting ?

Overfitting is a machine learning problem where a model learns the training data too well, including its noise, random fluctuations, and irrelevant details. As a result, it performs very well on the training data but poorly on new, unseen data.

In other words :

- Training accuracy : Very high
- Testing / Validation accuracy : Low

⇒ The model memorizes the data instead of learning the underlying patterns.

2. Why Does Overfitting Occur ?

Overfitting usually happens because :

① The model is too complex.
② The training dataset is too small.
③ The training data contains noise.
④ The model is trained for too many epochs.
⑤ Too many unnecessary features are included.

3. Characteristics of Overfitting

- Very low training error
- High testing error
- Poor generalization
- Large gap between training and validation accuracy

Overfitting is associated with high variance.

★ Underfitting : High bias, low variance
★ Good fit : Balanced bias and variance
★ Overfitting : Low bias, high variance

★ Bias : The difference between the model's average prediction and the true value.

★ Variance : Measures how much a model's predictions change when it is trained on different datasets.

6. Dropout (Neural Networks)

Dropout randomly disables a fraction of neurons during training.

Example :

Before: ○ ○ ○ ○ ○ ○

After Dropout (50%): ○ ✕ ○ ✕ ○ ✕

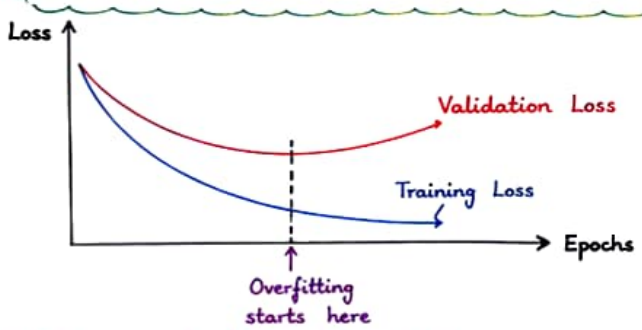
✕ = temporarily disabled

Benefits :

- Prevents neurons from relying too heavily on one another
- Improves generalization
- Common dropout rates : 0.2 – 0.5

4. How to Identify Overfitting ?

- Training accuracy ↑ but Validation / Test accuracy ↓
- Training loss ↓ but Validation loss ↑ after some epochs.



5. Techniques to Reduce Overfitting

① **Increase Training Data** :
More diverse data helps the model learn general patterns instead of memorizing specific examples.

② **Data Augmentation** (for images) :



③ **Simpler Model** :

- Use fewer layers
- Fewer neurons
- Lower polynomial degree
- Simpler algorithms

④ **Feature Selection** :
Remove unnecessary or irrelevant features.

Example (House Price Prediction)

Useful Features	Not Useful Features
• Area • Bedrooms • Location	• House owner's favorite color • Owner's phone number

⑤ **Regularization** : Add a penalty to the loss function.

L1 Regularization (Lasso)	L2 Regularization (Ridge)	Elastic Net
Loss = Original Loss + $\lambda \sum \lvert w \rvert$	Loss = Original Loss + $\lambda \sum w^2$	Combining L1 and L2 Regularization.
• Produces sparse models • Some weights become exactly zero • Useful for feature selection	• Shrinks weights toward zero • Rarely makes them exactly zero • Most commonly used in neural networks	

⑦ **Early Stopping** :
Stop training when validation performance stops improving.

Epoch	1	5	10	15
Training Loss	0.80	0.35	0.15	0.05
Validation Loss	0.85	0.40	0.25	0.27

Training loss keeps decreasing, but validation loss starts increasing after epoch 10.

Early stopping would stop training around epoch 10.

⑧ **Noise Reduction** :
Improve data quality by :

- Removing duplicate records
- Correcting incorrect labels
- Handling missing values
- Filtering out outliers when appropriate

⑨ **Hyperparameter Tuning** :
Adjust parameters such as :

- Learning rate
- Number of layers
- Number of neurons
- Batch size
- Regularization strength (λ)
- Dropout rate

Proper tuning improves generalization.

Key Takeaway :
Overfitting means the model learns noise and details instead of general patterns. Use the above techniques to build a model that generalizes well to unseen data.

UNDERFITTING

1. What is Underfitting ?

Underfitting occurs when a model is too simple to capture the underlying trend or pattern in the data.
It cannot learn the relationship well enough.

In other words :
- Training accuracy : Low
- Testing / Validation accuracy : Low

A statistical model or a machine learning algorithm is said to have underfitting when it cannot capture the underlying trend of the data.

It destroys the accuracy of our machine learning model.
It usually happens when we have fewer data to build an accurate model and also when we try to build a linear model with fewer non-linear data.

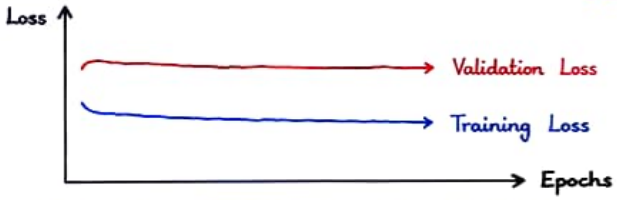
2. Key Characteristics

- High bias, low variance.
- Both training error and testing error are high.
- Model is too simple → cannot learn the complex pattern.
- Fails to capture the important relationships in data.
- Increasing training time or more data does not help much.

3. How to Identify Underfitting ?

- Training loss is high.
- Validation loss is also high.
- Training and validation accuracy are both low.
- Model predictions are far from actual values.

Both losses remain high and almost parallel.



No improvement even with more training.

4. Why Does Underfitting Occur ?

1. Model is too simple (low complexity).
2. Insufficient training data.
3. Important features are missing.
4. Strong regularization (too large λ).
5. Training time / epochs are too low.
6. High bias algorithms used for complex problems.

5. Techniques to Reduce Underfitting

① Increase Model Complexity
→ Use deeper networks, more layers, more neurons, higher polynomial degree, etc.

② Increase the Number of Features / Feature Engineering
→ Add relevant features or create new informative features.

③ Increase Training Data
→ More data helps the model learn the true pattern.

④ Train for More Epochs
→ Train longer until the model learns better.

⑤ Reduce Regularization Strength (smaller λ)
→ Excessive regularization can lead to underfitting.

⑥ Choose a More Powerful Algorithm
→ Use advanced algorithms that can capture complex patterns.

6. Underfitting in Bias - Variance Tradeoff

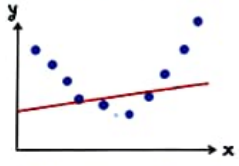
Model Fit	Bias	Variance
Underfitting	High	Low
Good Fit	Balanced	Balanced
Overfitting	Low	High

High bias means the model makes strong assumptions (too simple) and cannot fit the training data well.

7. Example (Polynomial Regression)

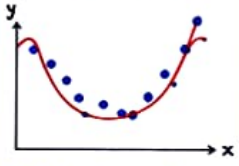
Suppose the true relationship between x and y is non-linear.
We fit models of different degrees.

Underfitting (Degree = 1)



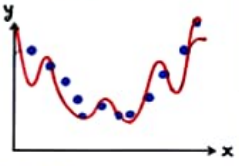
Model is too simple to capture the pattern. Both training and test errors are high.

Good Fit (Degree = 3)



Model captures the underlying trend well. Training and test errors are low.

Overfitting (Degree = 10)



Model learns noise as well. Training error very low but test error high.

8. Remedies Summary

- → Increase complexity
- → More relevant features
- → More data
- → Train longer
- → Reduce regularization
- → Use a better algorithm

9. Key Takeaway

Underfitting happens when the model is too simple to learn the data. It has high bias and low variance, resulting in poor performance on both training and testing data. Increase model complexity, provide more data, more features, and train longer to fix it.

Remember : Underfitting means the model hasn't learned enough. It sees the big picture but misses the details.

Training and Validation Curves

1. What are Training and Validation Curves ?

They are graphs that show how the performance of a model changes with the number of training epochs.

- Training curve → performance on training data.
- Validation curve → performance on unseen validation data during training.

> These curves help us understand whether the model is learning well, overfitting or underfitting.

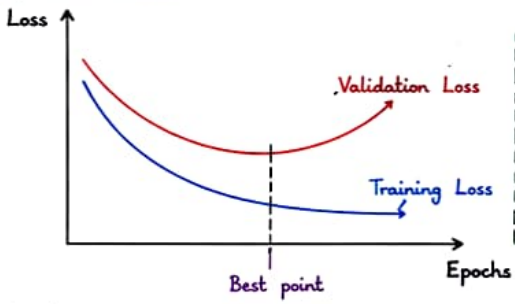
2. Why are they important ?

- Help to monitor the learning process.
- Detect underfitting and overfitting.
- Decide the right time to stop training (early stopping).
- Compare different models and hyperparameters.

> A good model has low training error AND low validation error with a small gap between them.

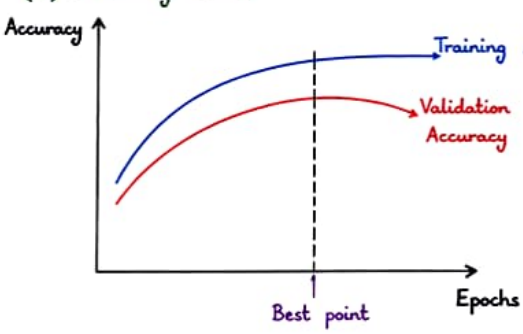
3. Plot : Common form of curves

(a) Loss curves



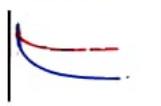
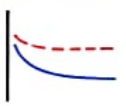
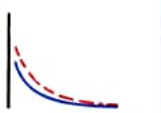
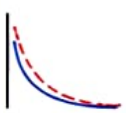
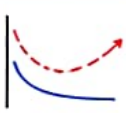
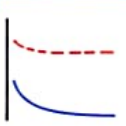
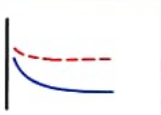
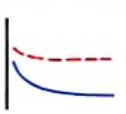
Validation loss starts increasing after some epochs while training loss keeps decreasing.

(b) Accuracy curves



Validation accuracy stops improving after some epochs and may decrease, while training accuracy keeps increasing.

4. Interpretation of Curves

Case	Training Curve	Validation Curve	Interpretation
1. Underfitting			• Both training and validation loss are high. • Model is too simple / not learned enough.
2. Good Fit			• Both training and validation loss decrease and remain close. • Best generalization.
3. Overfitting			• Training loss keeps decreasing but validation loss starts increasing. • Large gap → overfitting.
4. High Variance			• Training loss very low but validation loss very high and unstable. • Model is too complex.
5. High Bias			• Both losses high and close. • Model is very simple / cannot capture patterns.

(Blue → Training , Red dashed → Validation)

5. How to use these curves ?

- Train the model and plot both curves after each epoch.
- Choose the model at the best point (lowest validation loss / highest validation accuracy).
- Use early stopping : stop training when validation loss starts increasing for several epochs.
- Try different models / hyperparameters and compare curves.

Example (Loss vs Epochs)

Epoch	1	10	20	30	40	50
Training Loss	0.85	0.50	0.28	0.18	0.10	0.05
Validation Loss	0.90	0.45	0.30	0.32	0.38	0.50

Best epoch ≈ 20 (lowest val loss)

6. Summary

- Training curve → performance on data the model has seen.
- Validation curve → performance on unseen data.
- Small gap between them = good generalization.
- Large gap = overfitting or underfitting.
- These curves are essential for building a reliable model.

Remember : Always aim for the model that performs well on training data AND generalizes well to unseen data. ☆

DATA VISUALIZATION : Feature and Weight Visualization

1. Feature Visualization

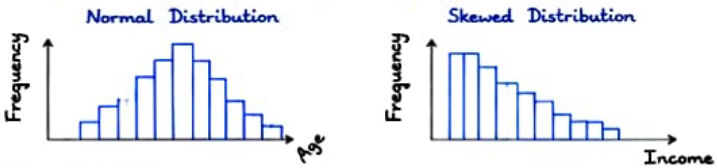
Feature visualization means representing the input data features in a visual form to understand their distribution, patterns, relationships and importance.

It helps in:
- Understanding the data better
- Detecting patterns and anomalies
- Choosing useful features
- Improving model performance

2. Ways to Visualize Features

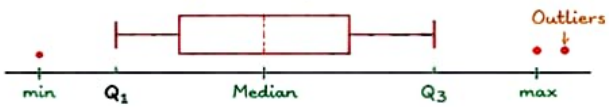
(a) Histogram (for individual feature)

Shows the distribution of a single feature.



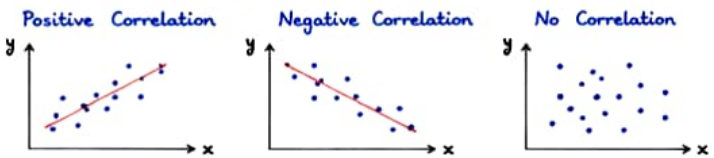
(b) Box Plot

Shows median, quartiles, spread and outliers.



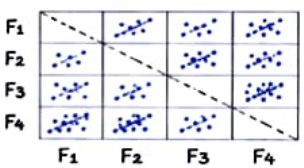
(c) Scatter Plot (for two features)

Shows relationship between two features.



(d) Pair Plot (Scatter matrix)

Visualizes pairwise relationships between many features.



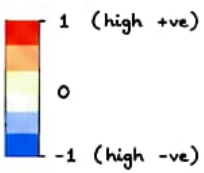
Diagonal → Histogram

Off-diagonal → Scatter plot

(e) Heatmap (Correlation Matrix)

Shows correlation between features using colors.

	F1	F2	F3	F4
F1	1.00	0.82	-0.1	0.30
F2	0.82	1.00	0.25	-0.20
F3	-0.1	0.25	1.00	-0.60
F4	0.30	-0.20	-0.60	1.00



1. Weight Visualization

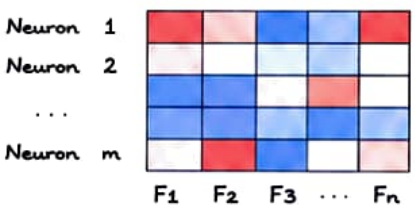
Weight visualization means visualizing the learned weights of a model (especially in neural networks) to understand what the model has learned.

It helps in:
- Understanding what patterns the model is detecting
- Debugging and model interpretation
- Finding dead or useless neurons / filters

2. Types of Weight Visualization

(a) Weight Matrix as Heatmap (for fully connected layer)

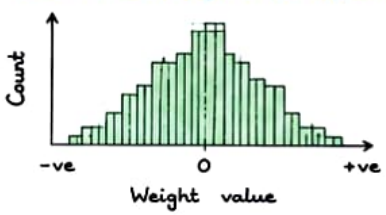
Rows → output neurons, Columns → input features



Red → +ve weight
White → near zero
Blue → -ve weight

(b) Weight Distribution (Histogram)

Shows how weights are spread.



If most weights are near zero, L2 regularization or sparse weights may be happening.

(c) Convolutional Filters Visualization

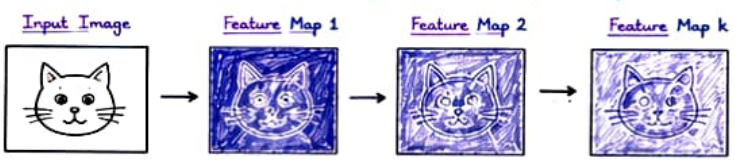
Each filter (kernel) in a CNN can be visualized as an image.

Example: First layer filters (3 × 3 × 3)



(d) Feature Maps (Activation Maps)

Shows what a filter is activating on in an input image.



3. Summary

- Feature visualization → helps us understand the data.
- Weight visualization → helps us understand what the model has learned.

Introduction to TensorFlow : Graphs, Nodes, Tensor Data Structures – Rank, Shape, Type, Building Neural Networks with TensorFlow

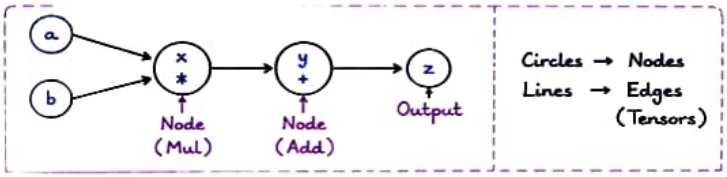
1. What is TensorFlow ?

- TensorFlow is an open-source machine learning library developed by Google.
- It is used to build and train machine learning and deep learning models.
- TensorFlow provides flexibility, speed and scalability.

2. Core Concepts : Graphs and Nodes

(a) Computational Graph

- TensorFlow represents computations as a graph.
- The graph consists of nodes (operations) and edges (tensors).



- a and b are input nodes.
- * and + are operation nodes.
- z is the output node.

(b) Node

- A node represents an operation (e.g., addition, multiplication, matmul, relu, etc.).

(c) Edge

- An edge represents a tensor flowing from one node to another.

3. Tensor Data Structures

A Tensor is a multi-dimensional array (similar to NumPy array) that flows through the graph.

(a) Rank (Number of dimensions)

Scalar	: Rank 0 (0-D tensor)	`5`
Vector	: Rank 1 (1-D tensor)	`[1, 2, 3, 4]`
Matrix	: Rank 2 (2-D tensor)	`[[1, 2, 3], [4, 5, 6]]`
3-D Tensor	: Rank 3 (3-D tensor)	`[[[1, 2], [3, 4]], [[5, 6], [7, 8]]]`

(b) Shape

- Shape is the size of each dimension.

Examples :

Tensor	Shape	Rank
5	()	0
[1, 2, 3, 4]	(4,)	1
[[1, 2, 3], [4, 5, 6]]	(2, 3)	2
[[[1,2],[3,4]],[[5,6],[7,8]]]	(2, 2, 2)	3

(c) Type (Data Type)

- TensorFlow supports various data types.

Common types :

- tf.int32
- tf.float32
- tf.float64
- tf.bool
- tf.string

In short : Rank = number of dimensions, Shape = size in each dimension, Type = kind of data in tensor.

4. Creating Tensors in TensorFlow

Using `tf.constant()`

```
import tensorflow as tf
a = tf.constant(10)                  # Rank 0
b = tf.constant([1, 2, 3, 4])        # Rank 1
c = tf.constant([[1, 2], [3, 4]])    # Rank 2
print(a.shape, a.dtype)
```

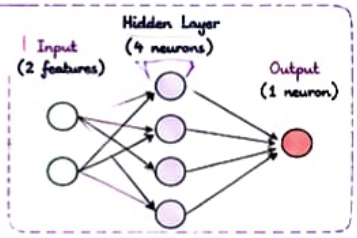
5. Building Neural Networks with TensorFlow (Keras API)

Keras is a high-level API in TensorFlow that makes building neural networks easy.

(a) Steps to build a Neural Network

① Import libraries
② Load / prepare data
③ Build the model (layers)
④ Compile the model
⑤ Train the model
⑥ Evaluate / Predict

(b) Example : Simple Feedforward Neural Network




```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# 1. Prepare data (Example)
x_train = [[0.,0.], [0.,1.], [1.,0.], [1.,1.]]
y_train = [[0.],   [1.],   [1.],   [0.]]

# 2. Build the model
model = keras.Sequential([
    layers.Dense(4, activation='relu', input_shape=(2,)),
    layers.Dense(1, activation='sigmoid')
])

# 3. Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# 4. Train the model
model.fit(x_train, y_train, epochs=500, verbose=0)

# 5. Evaluate
loss, acc = model.evaluate(x_train, y_train, verbose=0)
print("Accuracy:", acc)

# 6. Predict
predictions = model.predict(x_train)
print(predictions)
```

6. Summary

- TensorFlow uses graphs made of nodes (operations) and edges (tensors).
- Tensor is a multi-dimensional array with Rank, Shape and Type.
- Keras API helps to build, train and evaluate neural networks easily.
- Workflow : Data → Model → Train → Evaluate → Predict.

TensorFlow makes complex computations easy by organizing operations in graphs and working with tensors.